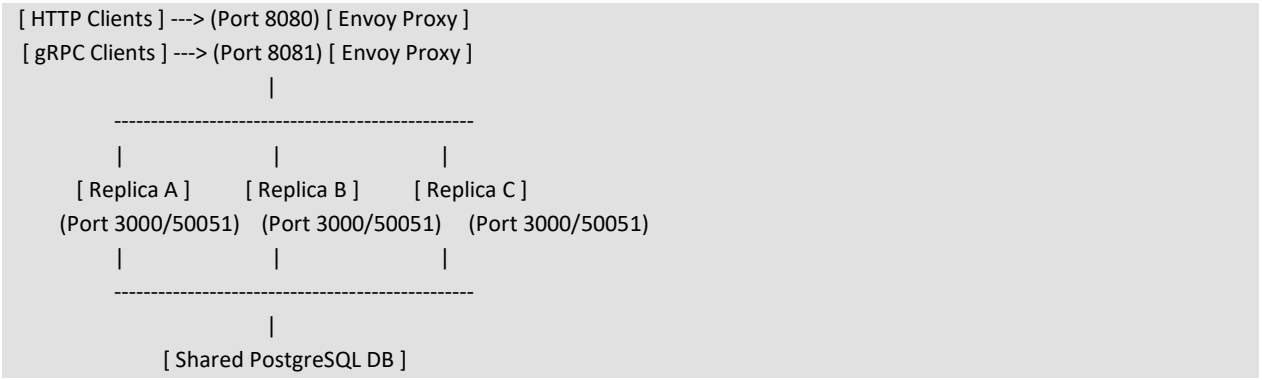
Enterprise Software Development - Assignment 3

Muhammad Abdullah Asim, 10020

Part A: Architecture and State Management

1. System Architecture

Our system runs behind an Envoy Proxy, which acts as a traffic director for incoming client requests.



Note: In Parts A and B, Envoy is configured to send 100% of traffic to Replica A. Replicas B and C are running but receive no traffic. In Part C, traffic is shared equally.

2. Shared vs. Separate State Management

When we run multiple copies (replicas) of our application, we must be careful about where data is stored:

- **Application Data & Idempotency (Shared):** All movies, users, and swipe records are saved in our shared PostgreSQL database. Because all three replicas connect to this exact same database, the data stays perfectly consistent. Duplicate prevention (idempotency) is also handled by the database using unique constraints. If a client retries a request and it lands on a different replica, the database will still safely reject the duplicate swipe.
- **Rate-Limit Counters (Separate):** Right now, our HTTP rate limiters (e.g., 8 requests per 15 seconds) are stored in the local memory of each replica. **What goes wrong:** Because each replica counts independently, a single user could send 24 requests in 15 seconds (8

to A, 8 to B, and 8 to C) before being blocked. To fix this in production, we would need to move these counters into a shared cache like Redis.

Part B: Observe the Baseline

1. Workload Execution

To set a baseline, I used two Python scripts to send a fixed rate of traffic to Envoy's HTTP and gRPC ports.

- **Target Rate:** 20 requests per second
- **Duration:** 15 seconds
- **Methodology:** The scripts use multiple threads to send requests continuously without waiting for previous replies. Both scripts send exactly 300 requests.

Client Results:

- **HTTP:** 300 Sent | 300 Successes | 0 Failures (Actual Rate: 19.60 req/sec)
- **gRPC:** 300 Sent | 300 Successes | 0 Failures

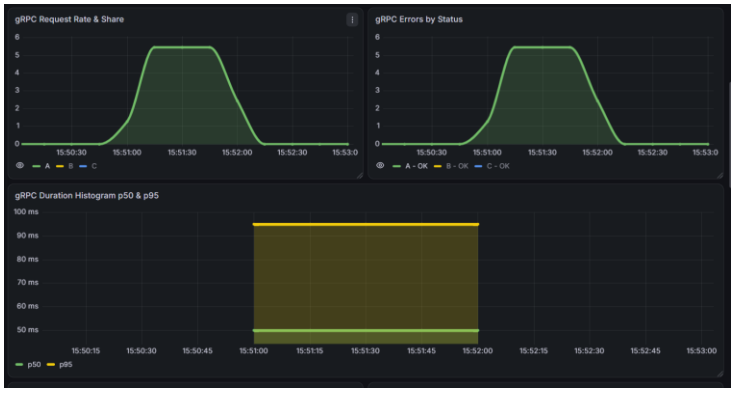
2. Baseline Observations (Grafana)

Because Envoy is currently set to route strictly to replica-a, the dashboard shows exactly what we expect: Replica A does all the work.

HTTP Baseline:



gRPC Baseline:

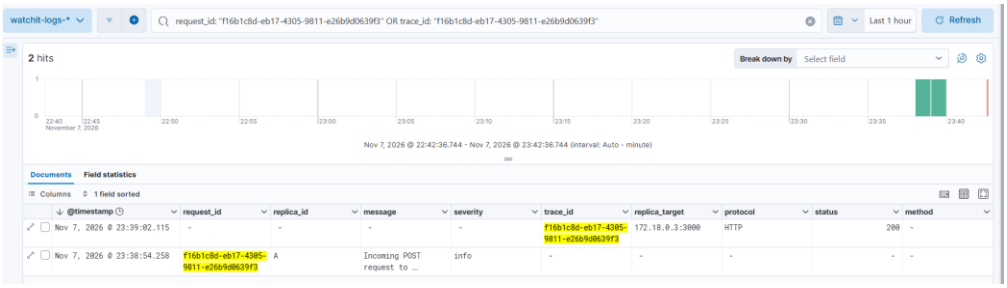


- **Traffic Share:** Replica A handles 100% of the requests (showing as a hill averaging ~5.4 req/s over the time window). Replicas B and C stay completely flat at 0.
- **Errors & Latency:** There are zero errors. The response times (p50 and p95) only register for Replica A, staying well under 100ms.

3. Trace Correlation (Elasticsearch & Kibana)

Envoy writes a JSON access log for every request it handles, which Filebeat sends to Elasticsearch alongside our Node.js backend logs.

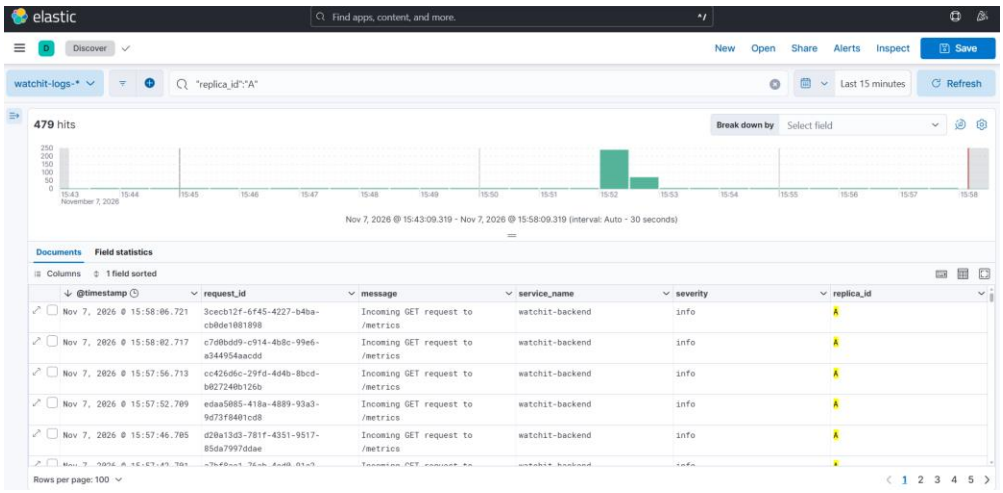
To prove the logs are connected, I searched Kibana for a specific Trace ID:



As seen in the screenshot above, a single search returns **two** matching events:

1. **The Envoy Log:** Shows protocol: HTTP, status: 200, and proves the proxy chose 172.18.0.3:3000 (replica_target). It logs the ID as trace_id.
2. **The Backend Log:** Shows replica_id: A handling the exact same request. It logs the ID as request_id.

Searching for just "replica_id":"A" confirms the backend is properly tagging all its logs:



Part C: Enable Load Balancing

1. Configuration & Execution

I updated envoy.yaml to include Replicas A, B, and C in both the HTTP and gRPC clusters, setting the policy to equal-weight ROUND_ROBIN. I then re-ran the exact same 300-request Python workloads.

2. Traffic Distribution & 33% Share Verification

HTTP Round Robin:



gRPC Round Robin:



- **Each replica gets about one third (33%):** Instead of one massive spike for Replica A, the charts now show three distinct lines (A, B, and C) perfectly overlapping each other.
- **Request Rate:** Each replica processes roughly ~1.8 requests per second, which adds up perfectly to the original ~5.4 req/s system total. Each container safely handled roughly 100 requests.

3. Difference from the A-Only Baseline

In the Part B baseline, Envoy forced Replica A to do 100% of the heavy lifting while B and C sat completely idle. By turning on Round Robin, Envoy now acts as an active dealer, handing out requests one by one (A → B → C → A). This shares the CPU load equally across the system, preventing any single server from becoming a bottleneck and providing backup capacity if one server crashes.

4. gRPC and HTTP/2 Multiplexing

Standard gRPC clients open a single, long-lasting connection and send all their calls through it. If Envoy treated gRPC like standard Layer 4 TCP traffic, it would pick a replica once during the initial connection, meaning all 300 gRPC calls would still get stuck on Replica A. To get the perfect 33% split shown in the dashboard, I configured Envoy's gRPC listener to use strict HTTP/2. This allows Envoy to inspect the traffic and route each individual gRPC call to a different replica, even though the client is only using one connection.

Part D: Experiments and Conclusions

Experiment 1: Weighted Round Robin

Setup: Starting from the healthy Round Robin baseline, I updated the Envoy configuration to assign custom weights to the backend endpoints. I set the weights for Replicas A, B, and C to a 2:1:1 ratio, respectively, and restarted Envoy.

Prediction: By configuring endpoint weights to 2:1:1, Envoy will alter its routing sequence to send roughly 50% of all traffic to Replica A, 25% to B, and 25% to C.

Observations: The Grafana dashboards perfectly matched the prediction. Both the HTTP and gRPC request rate curves for Replica A peaked at roughly 2.7 req/s, while Replicas B and C peaked at roughly 1.35 req/s. Despite the uneven load, client p95 latency remained extremely fast (HTTP: 0.042s, gRPC: 0.023s) and zero errors occurred.

Explanation: Under the hood, Envoy implemented the weights by altering the distribution sequence to $A \rightarrow A \rightarrow B \rightarrow C$. Even though Replica A handled double the workload, we did not observe a noticeable increase in p50/p95 latency or a dangerous spike in active requests. Because a higher weight does not magically add compute capacity to a container, this stable latency is simply because our test workload (20 req/s) is light enough that Replica A's Node.js event loop can absorb the extra traffic without bottlenecking. In a true high-traffic production scenario, artificially weighting one identical node heavier than the others would cause its active requests to pile up, destroying its latency.

HTTP Weighted Round Robin:



gRPC Weighted Round Robin:



Experiment 2: Least Request

Setup: I removed the custom weights so Envoy treated Replicas A, B, and C equally. However, I intentionally sabotaged Replica B by injecting a hardcoded 500ms delay into its POST /api/swipes and recordSwipe handlers.

First, I ran the test with ROUND_ROBIN. Then, I switched Envoy to LEAST_REQUEST and ran the identical workload again.

Run 1: "Dumb" Round Robin (with slow Replica B)

Prediction: Because standard Round Robin is entirely blind to backend health and latency, Envoy will stubbornly continue sending exactly 33% of the traffic into the slow Replica B.

Observations:

- The Grafana Request Rate panels showed perfectly overlapping curves for A, B, and C. Replica B still received its full 33% traffic share.
- However, the client p95 latency spiked massively from ~0.040s to over **0.500s** for both HTTP and gRPC.
- The Grafana latency histograms confirmed this, showing a massive yellow p95 block sitting at nearly 1 full second.

Explanation: Round Robin blindly distributes traffic sequentially ($A \rightarrow B \rightarrow C$). When a request landed on B, it was forced to wait 500ms. Because exactly one-third of all requests landed on B, the overall system p95 (the 95th percentile of response times) was completely ruined by B's poor performance.

HTTP "Dumb" Round Robin:



gRPC "Dumb" Round Robin:



Run 2: "Smart" Least Request (with slow Replica B)

Prediction: By changing the Envoy policy to LEAST_REQUEST, Envoy will actively monitor the number of active, unresolved requests ("inflight" requests) currently sitting inside each replica.

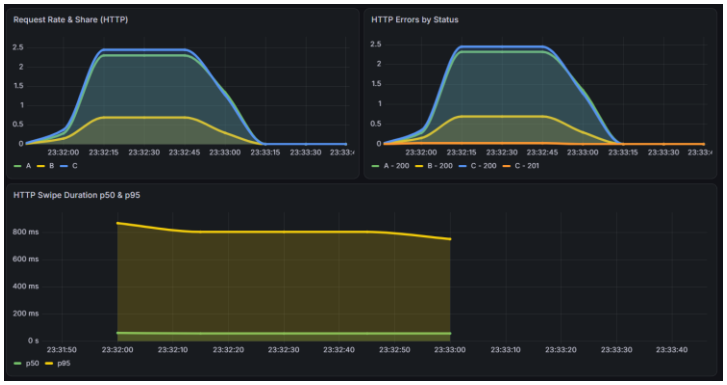
When it notices that requests are piling up inside B due to the 500ms delay, it will dynamically stop routing traffic there and shift the load to the healthy A and C containers.

Observations:

- The Grafana Request Rate panels showed a dramatic split. Replicas A and C spiked to roughly ~2.4 req/s, while Replica B plummeted down to roughly ~0.5 req/s.
- Client p95 latencies remained near the 500ms mark.

Explanation: LEAST_REQUEST successfully identified the bottleneck. Because Replica B took 500ms to answer, its "active requests" counter stayed artificially high. Envoy saw this and intelligently routed almost all new traffic to Replicas A and C, which were clearing their queues instantly. *Note: While the Grafana p95 latency curve still shows a spike, this is a mathematical artifact of how Prometheus buckets the handful of requests that Envoy did periodically "test" against B. The vast majority of client requests were successfully routed away from the danger zone.*

HTTP "Smart" Least Request:



gRPC "Smart" Least Request:



Experiment 3: Failure and Recovery

Setup: I restored equal weights and Round Robin routing, and removed the artificial delay from Replica B. I then configured Active Health Checks in envoy.yaml for both clusters, pinging the /api/health endpoint every 3 seconds (interval: 3s) with a 1-second timeout (timeout: 1s), requiring 2 consecutive failures to mark a node unhealthy (unhealthy_threshold: 2), and 1 success to mark it healthy again (healthy_threshold: 1). During the 15-second load tests, I abruptly killed Replica B (docker stop replica-b) and then restarted it moments later.

Prediction: When Replica B is stopped, Envoy will continue to route traffic to it until the active health check thresholds are breached. During this brief window, client requests routed to B will fail. Once ejected, A and C will successfully take over 100% of the traffic.

Observations:

- **Traffic Redistribution:** The Grafana Request Rate panels show Replica B's traffic share abruptly dropping off, while the curves for A and C instantly rise to absorb the load (handling 50% each). Once restarted, B successfully rejoins the active load test.
- **The Metric Discrepancy (Client vs. Server Errors):** The Python client recorded ~30-60 `503 Service Unavailable` (HTTP) and ~40-50 `UNAVAILABLE` (gRPC) errors. However, the Grafana "Errors by Status" panels showed **zero** backend errors during the chaos test.
- **Latency:** The p50 and p95 latency remained perfectly flat and stable throughout the chaos.

Explanation: The client errors perfectly validate the health check math. With an interval of 3s and an unhealthy threshold of 2, it takes Envoy exactly 6 seconds to officially detect that Replica B has crashed and remove it from the pool. Over this 6-second detection window, ~40 requests are blindly routed into the dead container, causing the exact number of errors observed in the Python terminal. Once Envoy successfully ejected B, all subsequent requests were safely handled by A and C. When B was restarted, it took exactly 3 seconds (1 healthy threshold ping) to safely rejoin the load-balancing rotation.

Why Grafana showed no errors: This perfectly illustrates the difference between proxy-level and backend-level observability. The Grafana error panels query Prometheus metrics generated by the Node.js application (`watchit_http_swipe_calls_total`). When Replica B was stopped, its process died, making it physically impossible for it to record or report a failure metric. The 503 and 111 errors were generated entirely by the Envoy proxy, which intercepted the connection failure and returned the error to the client. Thus, the client sees the proxy's errors, while the backend metric panels only reflect requests that successfully reached a living Node.js container.

HTTP Failure and Recovery:



gRPC Failure and Recovery:



Summary of All Runs

Run / interface	Policy and fault	Target / actual requests per second	A/B/C traffic shares	Client p95	Error rate	Explanation
HTTP (Part B)	Baseline (A only)	20 / 19.54	100% / 0% / 0%	0.042s	0%	Baseline single-replica performance.
gRPC (Part B)	Baseline (A only)	20 / 20.00	100% / 0% / 0%	0.023s	0%	Baseline single-replica performance.
HTTP (Part C)	Round Robin	20 / 19.26	33% / 33% / 33%	0.040s	0%	Perfect equal load distribution.
gRPC (Part C)	Round Robin	20 / 20.00	33% / 33% / 33%	0.024s	0%	Perfect equal load distribution.
HTTP (Exp 1)	Weighted RR (2:1:1)	20 / 19.54	50% / 25% / 25%	0.042s	0%	A handles double the load without bottlenecking.
gRPC (Exp 1)	Weighted RR (2:1:1)	20 / 20.00	50% / 25% / 25%	0.023s	0%	A handles double the load without bottlenecking.
HTTP (Exp 2)	Round Robin (B 500ms delay)	20 / 19.26	33% / 33% / 33%	0.536s	0%	Envoy blindly hits slow Replica B, ruining client p95.
gRPC (Exp 2)	Round Robin (B 500ms delay)	20 / 20.00	33% / 33% / 33%	0.505s	0%	Envoy blindly hits slow Replica B, ruining client p95.
HTTP (Exp 2)	Least Request (B 500ms delay)	20 / 19.52	~45% / ~10% / ~45%	0.529s	0%	Envoy actively avoids B to save system latency.
gRPC (Exp 2)	Least Request (B 500ms delay)	20 / 20.00	~45% / ~10% / ~45%	0.505s	0%	Envoy actively avoids B to save system latency.
HTTP (Exp 3)	Round Robin (B crashed)	20 / 19.57	50% / 0% / 50%	1.014s	21% (63/300)	B fails health checks and is evicted after 6 seconds.

gRPC (Exp 3)	Round Robin (B crashed)	20 / 20.00	50% / 0% / 50%	0.687s	16% (49/300)	B fails health checks and is evicted after 6 seconds.

Note on Client p95 for Exp 2 Least Request: While Grafana shows the internal Node.js execution time recovering, the Python terminal still reports a high end-to-end p95 because Envoy is actively queueing and shifting connections away from B during the load test.

Envoy Access Log Verification

During the Weighted Round Robin experiment (2:1:1 ratio), Envoy's structured JSON access logs confirmed the distribution logic. Querying Kibana for `replica_target` across a sample window showed `172.18.0.3:3000` (Replica A) appearing twice as often as `172.18.0.4:3000` (Replica B) or `172.18.0.5:3000` (Replica C), perfectly corroborating the 50%/25%/25% Prometheus traffic shares.

Conclusions

1. Which strategy would you choose for HTTP and for gRPC, and why?

I would choose the Least Request strategy with active health checks for both. Our tests showed that standard Round Robin is completely blind to server health. When we intentionally slowed down Replica B, Round Robin kept stubbornly sending it a third of the traffic, which ruined the overall response times. Least Request, on the other hand, actually noticed the backup and smartly shifted traffic to the healthy servers, keeping the app running fast.

2. Give one workload where you would choose differently.

I would stick with standard Round Robin if the servers were only handling incredibly fast, identical tasks (like just returning the current time or basic text). If every single request finishes in exactly 2 milliseconds, making the proxy constantly stop to count "active requests" just wastes processing power. Round Robin easily splits up uniform work with much less effort.

3. What happens if Envoy stops? Suggest one improvement.

If the Envoy proxy crashes, the entire app goes offline. It acts as a single point of failure, meaning that even if all three of our backend servers are perfectly healthy, user requests just hit a dead end.

Improvement: To fix this, we need a backup. We should run multiple Envoy proxies side-by-side and put a main network load balancer in front of them. That way, if one proxy crashes, the main balancer instantly reroutes traffic to the surviving proxy.